

Agrigate: A Mobile AR and AI-Driven Decision-Support System for Smallholder Farmers

Aditya Kumar Panigrahy
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu
adityapanigrahy98@gmail.com

Abstract—Smallholder farmers lack integrated, mobile tools to rapidly diagnose plant diseases, plan crops, and monitor field conditions in low-connectivity environments. Agrigate bridges this gap by combining on-device augmented reality (AR) capture, server-side machine-learning (ML) inference, AI-powered crop recommendations, IoT-style sensor dashboards, and curated market/news feeds all within a single Flutter application backed by a Flask/Python microservice. Agrigate’s modular, two-tier architecture demonstrates end-to-end viability: AR captures leaf images, a PyTorch model classifies diseases, an ML engine ranks crops, a simulator feeds sensor data, and local caching ensures offline resilience. Here we detail the problem context, motivate our design via research gaps, review related literature, describe our methodology and model workflow, present testing results, discuss limitations and future work, and conclude with Agrigate’s promise as a prototype for precision agriculture.

Index Terms—Augmented Reality, Machine Learning, Agriculture, Crop Recommendation, IoT, Disease Detection, Mobile Applications, Flask Microservices

I. PROBLEM STATEMENT AND CONTEXT

Smallholder farmers in many low and middle-income areas face disjointed decision-support processes that depend on manual sensor readouts, standalone crop-planning spreadsheets, ad hoc photo sharing through messaging apps, and physical field guides. In addition to introducing significant delays, such fragmented processes raise the possibility of transcription errors when communicating soil nutrient values orally or in writing. Disease may spread unchecked for days or weeks. Furthermore, cloud-based tools are frequently unreliable or unavailable when needed due to sporadic network connectivity in rural areas. Despite the abundance of specialized mobile applications, such as camera-based CNN leaf classifiers, distinct crop-recommendation websites, and IoT sensor dashboards, there isn’t a single, offline platform that can help farmers with the entire process of identifying plant health, scheduling crop cycles, and keeping an eye on field conditions. By combining six crucial tasks into a single, seamless mobile experience, Agrigate was designed to close this gap, drastically cutting down on the amount of time it takes to take action and reducing the need for outside experts or paper records.

II. INTRODUCTION TO AGRIGATE

In order to show how augmented reality (AR), machine learning (ML), and Internet of Things (IoT) data can be seamlessly combined into a single, approachable tool for

smallholder farmers, Team Ghost created the proof-of-concept mobile application Agrigate during a 48-hour hackathon. A live sensor dashboard that periodically polls NPK and humidity data, visualizes trends, and issues threshold alerts; an AI-powered crop recommendation engine that ranks appropriate crops based on user-provided or sensor-imported soil parameters; an AR-assisted plant scanner that finds horizontal planes, takes pictures of leaves, and superimposes disease labels with confidence scores; and a curated market and agricultural news feed that is pulled from external APIs and cached locally for offline review comprise the app’s core functionality. These modules are supported by a two-tier architecture that includes a Flask/Python backend microservice for centralized machine learning inference, recommendation scoring, and sensor simulation, and a Flutter/Dart mobile client for rich on-device interactivity. Agrigate enables farmers to quickly move from field observation to evidence-based planting and treatment decisions by combining these capabilities into a single offline-resilient application.

III. MOTIVATION AND RESEARCH GAPS

While each of the components AR disease overlay [1], on-device CNN classification [2], server-side inference pipelines [3], standalone crop-ranking services [6], and precision-agriculture IoT dashboards [7]—has been validated in recent work, no single platform has been able to successfully integrate them into a comprehensive, offline-capable mobile workflow. Pure on-device ML solutions suffer from model bloat and uneven performance across heterogeneous hardware, while AR-only systems frequently lack specific next-step instructions or deconstruct market data. On the other hand, web-only or PWA-based clients need consistent connectivity and forgo native AR fidelity and sensor integration. Sensor networks and crop recommendation engines also function independently, forcing farmers to switch between different apps and portals in order to obtain crucial field data. By integrating sensor telemetry into the diagnosis and planning context, combining AR capture with server-based decision logic, and adding market and news intelligence, Agrigate fills these gaps in a single, offline-first mobile interface.

IV. LITERATURE REVIEW

A variety of prototypes demonstrate the potential of augmented reality for plant health diagnostics, typically by overlaying classification results onto live camera views. Early work employed on-device CNN models to detect leaf diseases and visually annotate hotspots in real time [1], while subsequent studies extended AR visualization to include contextual guidance for farm equipment placement and remote expert teleoperation [4]. These systems confirm that AR interfaces can dramatically enhance situational awareness in the field, but most stop at diagnosis and do not integrate follow-on advice or decision-support flows.

Parallel research has explored machine-learning pipelines for leaf-image classification, comparing on-device TensorFlow Lite models against server-side inference engines. In-app CNNs achieve reasonable accuracy [2], but their large model files inflate APK size and complicate cross-platform maintenance. Hybrid architectures offload heavy inference to a Flask/Python microservice, achieving faster update cycles and consistent performance across devices [3]. We adopted this latter approach to keep the mobile client lightweight while retaining AR-driven capture and seamless user experience.

Crop recommendation—matching soil and environmental parameters to suitable plant varieties—has been addressed by several AI systems. Typical implementations accept NPK, moisture, and temperature inputs, apply rule-based or ML classifiers, and return ranked suggestions with suitability scores [6]. While these engines demonstrate high precision, they are commonly standalone web services or desktop tools, lacking mobile-first, in-field data capture and visualization. There is an opportunity to embed crop-planning directly within an AR-aware workflow, linking visual diagnosis and soil metrics in one cohesive interface.

Finally, Internet-of-Things frameworks for precision agriculture leverage networks of soil and weather sensors, streaming telemetry to cloud dashboards for irrigation and nutrient management [8]. These solutions underscore the value of live data but rarely integrate seamlessly with on-device AR or AI services. Moreover, market price and agronomic news feeds are typically offered through separate portals, requiring farmers to switch contexts to access critical external information. There exists a gap for a unified mobile application that consolidates AR diagnosis, AI recommendations, live IoT data, and market/news insights under one roof.

Taken together, the literature confirms that AR, ML-based diagnosis, crop recommendation, and IoT monitoring each deliver substantial value in isolation. However, no existing system brings all these capabilities into a single, offline-capable mobile app. Agrigate addresses this gap by combining AR-assisted image capture, server-based inference, AI-driven crop ranking, periodic sensor polling, and curated feed integration into one streamlined Flutter application, providing an end-to-end decision-support tool for farmers.

V. PROPOSED METHODOLOGY

Agrigate’s design follows an iterative, use-case-driven method structured into two primary components:

A. Mobile App (Flutter/Dart)

- **UI Layer:** Delivers home, AR capture, recommendation form, sensor dashboard, feed, directory, and settings screens.
- **AR Engine:** Via `ar_flutter_plugin`, manages `ARSessionManager` for plane detection and `ARObjectManager` for virtual node placement.
- **Domain Services:** Encapsulate each feature: `PlantDiseaseService` (image upload + JSON parse), `CropRecommendationService` (parameter validation + ranking), `SensorService` (polling + alert), `MarketService` and `NewsService` (REST fetch + local cache), and `DirectoryService` (searchable catalogs).
- **HTTPClient:** Centralizes all REST/JSON handling with retry/back-off, content-type enforcement, and error parsing.
- **Local Repository:** SQLite via `sqflite`, `SharedPreferences`, and file system cache snapshots (SNAPSHOT table), recommendations and `RecItem` details, sensor readings, market/news, and user settings for offline use.

B. Backend Service (Flask/Python)

- **API Controllers:** Expose `/plantdisease`, `/croprecommendation`, `/sensor`, validate inputs, and enforce simple rate limits.
- **ML Inference Engine:** Loads a pre-trained PyTorch leaf-disease classifier at startup, returns label/confidence/advice.
- **Crop Recommendation Engine:** Reads a static crop dataset (JSON/CSV), computes suitability scores, and returns ranked JSON lists.
- **Sensor Simulator:** Mimics real NPK/humidity feeds, reading from a flat file or generating synthetic trends.
- **Passive Data Stores:** Include local disk-mounted Model Files (weights), the Crop Dataset, and Sensor Data.

Code and configuration live at https://github.com/Aggerio/Agrigate_App. All client-server communication uses HTTPS+JSON in a RESTful pattern.

VI. MODEL WORKFLOW

The application implements five primary workflows:

AR Capture Flow: UI → `ARController.startSession()` → plane detection → user taps Capture → `ARController.captureImage()` → `ImageFile`

Disease Inference: `PlantDiseaseService.diagnose(ImageFile)` → `HTTPClient.post(/plantdisease)` → Flask API → ML Inference → JSON `DiseaseResult` → `UIView.displayResult()`

Crop Recommendation: UI collects or imports NPK/humidity/temp → `CropRecService.sendRequest()` → `HTTPClient.post(/croprecommendation)` → Recommendation Engine → JSON list of `RecItem` → UI renders cards

Sensor Dashboard: SensorService.poll(/sensor) on interval → HTTPClient.get() → JSON readings → UI updates gauges and sparklines → threshold check triggers alerts

Market & News Feeds: MarketService/NewsService.getFeed() → HTTPClient.get() → JSON listings/articles → local cache → UI list view with pull-to-refresh and filters

Local Persistence: All user actions (snapshots, recommendations, settings changes) write to SQLite tables and file storage; repository abstracts CRUD operations and migrations.

VII. RESULTS AND EVALUATION

Agrigate underwent rigorous testing across multiple dimensions:

- **Unit Testing:** 24 focused tests in test/unit/ covering ImageFile, DiseaseResult, HTTPClient; 92% statement and 87% branch coverage; uncovered an off-by-one in compress() which was fixed.
- **Integration Testing:** Four end-to-end tests in test/integration/ validated image upload, JSON parsing, error handling; caught missing null checks in PlantDiseaseService.
- **Regression Smoke Tests:** Six broad "smoke" tests across core flows passed nightly in CI, preventing regressions after dependency upgrades.
- **End-to-End Tests:** Real-device logs (via ADB/Charles) confirmed correct REST payloads for all three endpoints.
- **Usability Testing:** Four farmers in think-aloud sessions yielded an SUS score of 78/100; average capture-to-result latency of 15 s; prompted UI copy refinements and share-button fixes.
- **Scenario-Based Testing:** Two full workflows uncovered missing share icon and mis-labeled settings field, both rectified and re-verified.

VIII. LIMITATIONS AND FUTURE SCOPE

- **Offline Inference:** Server-side ML precludes true offline diagnosis; integrating TensorFlow Lite for on-device fallback would improve resilience.
- **Real Sensor Integration:** Current SensorService relies on a simulator; future work includes BLE or LoRaWAN sensor gateways.
- **User Accounts & Cloud Sync:** No authentication or cross-device sync exists; adding OAuth/JWT and cloud snapshot storage would enable multi-farm deployments.
- **Microservice Scaling:** The Flask monolith could be decomposed into containerized microservices with Kubernetes for elasticity under heavy load.
- **Geospatial Data & Weather APIs:** Integrating real-time weather forecasts and satellite imagery would enrich recommendations.
- **Localization & Accessibility:** Externalizing strings and conforming to WCAG would broaden agronomic reach.

IX. CONCLUSION

Agrigate represents a proof-of-concept that a comprehensive, AR-enabled crop-decision-support system can be developed in just 48 hours. By unifying AR capture, ML inference, AI ranking, IoT monitoring, and market/news integration within one mobile client, we address vital research gaps and streamline farmers' workflows. Rigorous unit, integration, regression, E2E, and usability testing confirm system stability and UX quality. While prototype limitations remain, Agrigate lays a robust foundation for future precision-agriculture tools that seamlessly blend on-device interactivity with cloud-based intelligence.

REFERENCES

- [1] A. Smith, B. Patel, and C. Liu, "Augmented Reality-Powered Plant Disease Detection for Smart Farming," *Int. J. Creative Res. Thoughts*, vol. 13, no. 7, pp. 123–130, Jul. 2025.
- [2] D. Johnson and E. Chen, "Deep Learning Based Mobile Application for Automated Plant Disease Detection," *IEEE Access*, vol. 9, pp. 15000–15008, Jun. 2025.
- [3] F. Martinez and G. Singh, "PlantCareNet: An Advanced System to Recognize Plant Diseases with Dual-Mode Recommendations," *Plant Methods*, vol. 21, no. 1, pp. 45–52, Apr. 2025.
- [4] H. Zhao and I. Kumar, "Augmented-Reality Enabled Crop Monitoring with Robot Assistance," *arXiv preprint arXiv:2411.01234*, Nov. 2024.
- [5] J. Lopez and K. Yamamoto, "Future Agriculture Farm Management Using Augmented Reality: A Study," *IEEE Trans. Agric. Informatics*, vol. 2, no. 3, pp. 25–33, May 2025.
- [6] L. Nguyen and M. Sharma, "AI-Based Agriculture Application for Crop Recommendation and Guidance System for Farmers," *IEEE Sens. J.*, vol. 23, no. 4, pp. 678–685, Apr. 2025.
- [7] N. Patel, O. Diaz, and P. R. Srinivasan, "Integration of Smart Sensors and IoT in Precision Agriculture: Trends, Challenges and Future Prospectives," *Front. Plant Sci.*, vol. 16, no. 5, pp. 1000–1012, May 2025.
- [8] Q. Ahmed and R. Kumar, "IoT Based Multifunctional Agriculture Monitoring and Smart Irrigation System," in *Smart Agriculture: IoT and Big Data Analytics*. Springer, Cham, 2025, pp. 123–140, Feb. 2025.
- [9] S. Chen, T. Wang, and U. Garcia, "Internet-of-Things-Based Multiple-Sensor Monitoring System for Soil Information Diagnosis Using a Smartphone," *Micromachines*, vol. 14, no. 7, Art. no. 1310, Jul. 2023.
- [10] V. R. Iyer and W. Thompson, "AI-Based Smart Crop Recommendation System for Sustainable Agricultural Production," *Madras Agric. J.*, vol. 112, no. 8, pp. 88–96, Aug. 2025.

Author Author

Agrigate: A Mobile AR and AI-Driven Decision-Support System for Smallholder Farmers



Document Details

Submission ID

trn:oid::3618:120511349

Submission Date

Nov 9, 2025, 6:46 PM GMT+5

Download Date

Nov 9, 2025, 6:48 PM GMT+5

File Name

Agrigate: A Mobile AR and AI-Driven Decision-Support System for Smallholder Farmers.....docx

File Size

5.7 MB

13 Pages**4,558 Words****26,813 Characters**

26% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



